

LAMP

LATEX // APACHE // MARIADB // PAIN

From a Path Header to Container-Root RCE Through XeLaTeX

TEX INJECTION // PATH HEADER // SHELL ESCAPE // RCE

AUTHOR

Denis Krüger

PROJECT

LAMP Security Research

IMPACT

Container-Root RCE

ENVIRONMENT

FAUST CTF 2026

FOCUS

Exploit / Analyze / Automate

REPORT DATE

September 28, 2026

Abstract

This report documents the analysis and exploitation of the *LAMP* service during FAUST CTF 2026.

The service—appropriately expanded as *LaTeX*, *Apache*, *MariaDB*, *Pain*—implemented significant portions of its HTTP processing and application logic using XeLaTeX.

The investigation identified multiple weaknesses across the service’s request parsing, TeX processing, persistence, and rendering mechanisms. Individually, these weaknesses exposed constrained primitives including arbitrary file access and stored TeX injection. When chained together, they enabled operating system command execution as root within the LAMP service container.

A 32-byte limit on the primary injection field initially constrained the usefulness of the command-execution primitive. The restriction was subsequently bypassed by abusing newline handling in persisted ship names, allowing attacker-controlled files within the application’s storage directory to act as shell-command carriers.

The completed exploit chain was operationalized during the live Attack/Defense competition, used to recover service flags from opposing teams, and integrated with the team’s automated flag-submission infrastructure.

Contents

Abstract **i**

1 What the Hell Is LAMP? **1**

 1.1 Why XeLaTeX Matters 1

 1.2 And Then There Was Shell Escape 1

 1.3 The Exploit Chain 2

 1.4 What This Writeup Covers 2

1 What the Hell Is LAMP?

LAMP was one of the services deployed during FAUST CTF 2026.

From the outside, it behaved like a fairly normal web application: users could register accounts, create ships, add components, and later view the resulting state through HTTP requests.

The implementation was considerably less normal.

Large parts of the service were processed through XeLaTeX.

LaTeX // Apache // MariaDB // Pain

That last component became increasingly accurate as the analysis continued.

1.1 Why XeLaTeX Matters

LaTeX is normally used as a document preparation system. A source file contains text together with commands, and a TeX engine interprets those commands to produce a document.

A minimal example looks like this:

```
\documentclass{article}

\begin{document}
Hello.
\end{document}
```

The important security detail is that TeX commands are not merely formatting markers.

TeX includes macros, token expansion, file input and output, dynamic control sequences, and other behaviour that makes it much closer to a small programming environment than to a passive markup format.

That becomes interesting when values controlled by an HTTP client stop being treated as ordinary text and instead become TeX source.

LAMP did exactly that in several places.

1.2 And Then There Was Shell Escape

The service started XeLaTeX using:

```
xelatex -8bit --shell-escape /srv/main.tex
```

The important part is:

--shell-escape

Shell escape allows TeX functionality to invoke external operating-system commands.

This is not automatically a vulnerability. If all TeX input is trusted, it may be an intentional feature.

LAMP, however, processed attacker-controlled HTTP and database values through that same TeX environment.

FINDING //

The interesting question was therefore not whether shell escape existed. The question was whether user-controlled data could be transformed into executable TeX before it reached a shell-enabled primitive.

1.3 The Exploit Chain

The final exploit did not come from a single bug.

Instead, several smaller weaknesses could be chained together:

1. A client-controlled `Path` header could overwrite the path used by the application after the original request path had already been validated.
2. Component metadata stored in MariaDB was later inserted back into the generated TeX and interpreted during rendering.
3. The useful injection field was limited to 32 bytes, restricting the commands that could be executed directly.
4. A separate newline injection in persisted ship data provided a place to store a much longer shell command.

Together, these behaviours provided:

arbitrary file access → stored TeX injection → command execution → flag harvesting

The resulting command execution ran as root inside the LAMP service container.

Throughout this writeup, this is therefore described as *container-root RCE*. The exploit did not by itself demonstrate root access to the underlying Vulnbox host.

1.4 What This Writeup Covers

This is not intended to be a complete study of TeX internals.

The goal is much simpler:

- document the code paths that mattered during the competition;
- explain the XeLaTeX behaviour required to understand the exploit;
- reconstruct how the individual primitives were chained;
- preserve the payloads and reasoning behind the final PoC;
- and document how the exploit was automated during the live CTF.

The next section begins with the first useful primitive:

convincing LAMP to read a path it had never actually validated.